

COMS W4156 Advanced Software Engineering (ASE)

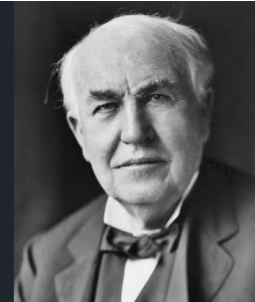
October 22, 2024

On this day ...



Edison Perfects Light Bulb

1879 Thomas Edison perfects carbonized cotton filament light bulb



Inventor
Thomas Edison

Agenda

1. An Ending and a Beginning
2. Upstream Software Engineering
3. User Stories
4. Use Cases
5. Reminders



First Assessment Coming Soon to a Computer near You

Available this Friday October 25, 12:01AM (you probably consider this Thursday night)

Due Monday October 28, 11:59PM

See last Thursday's lecture (Oct 17) for details about the ungraded questions

Submission: You must submit a pdf file on courseworks. You can submit as many times as you like until the deadline. Submit early, submit often

The assessment is
of your total grade

10%



Late Policy



Submissions posted within one day late (which includes everything courseworks marks as 1 minute late up to 23 hours 59 minutes late) will receive maximum 75% credit; within two days late maximum 50% credit; within three days late maximum 25% credit; more than three days late 0 credit

What this means in real life: We will grade your assignment normally (assuming you completed 0A). If you submit on time, that's the grade you get (even if courseworks says you were late by 0 minutes). If you submit the next day (which starts at 12:00AM), your score is multiplied by 75% (0.75). If you submit the day after that, i.e., within two days late, your score is multiplied by 50% (0.5). If you submitted within three days late, your score is multiplied by 25% (0.25). Any later, you get 0

Second Iteration Begins After Assessment Deadline

[Second Iteration](#) due November 25 (Monday before Thanksgiving)

[Second Iteration Demo](#) due December 6 (the Friday a week after Thanksgiving)

Second Assessment Friday December 6 12:01AM through Monday December 9 11:59PM (last day of classes and the day before university "study days")

[Demo Day](#) Monday December 16 10AM to 4PM (Monday of finals week)



(a)



(b)

Agenda

1. An Ending and a Beginning
2. Upstream Software Engineering
3. User Stories
4. Use Cases
5. Reminders



Upstream vs. Downstream Software Engineering



Our emphasis so far has been on "downstream" SE: implementing features, finding and fixing bugs, first in someone else's code (individual miniproject), then in your own code (team project)

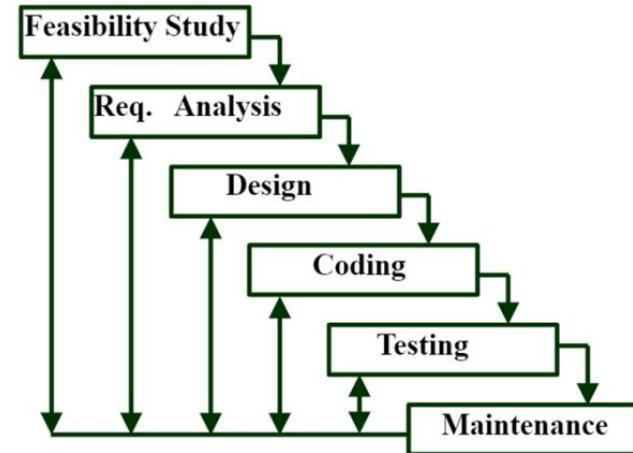
There will be additional downstream material in the second half of the course, as you develop a client for your team's service, but now we will also consider "upstream" SE: software process and planning, requirements and design

Software Process Models: Waterfall

The "[waterfall](#)" model separates development into sequential phases, where one phase must be completed (and reviewed and approved) before starting the next phase. For example, the design of the entire software product must be completed before starting to code, and all the code must be written before starting to test. This is easy to explain to a CS1 student, C-suite management, government procurement agent, etc.

When problems are found during one phase, they may be due to a flaw or omission at a previous phase, but backtracking can cause > 100% budget and schedule overrun

Adding [feasibility study](#) (or [design sprint](#)) at beginning reduces but does not eliminate risk



Software Process Models: Agile

WATERFALL



Nowadays most software development follows an "[agile](#)" model, where a minimum viable product ([MVP](#)) is usually completed first, but thereafter features are added incrementally in a series of iterations (aka sprints) and released frequently - possibly even after every iteration

Although the metaphor is broken, "upstream" vs "downstream" are part of agile. We still need to do requirements and design, but they are revisited every iteration. EXCEPT certain kinds of requirements must be completed up front - specifications like [HTTP](#), [URL](#) RFCs are necessary for interoperability across large numbers of different vendors, e.g., [so internet software can talk to each other](#) (formal requirements outside scope of this class)

Agenda

1. An Ending and a Beginning
2. Upstream Software Engineering
3. User Stories
4. Use Cases
5. Reminders

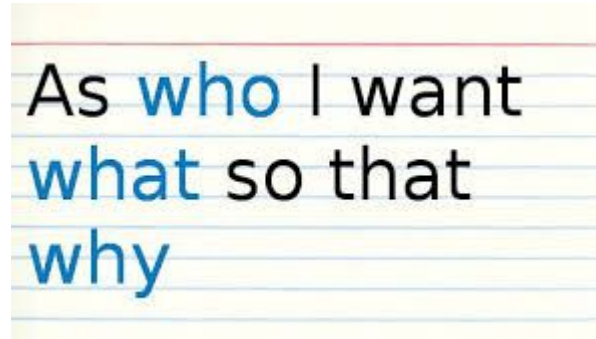
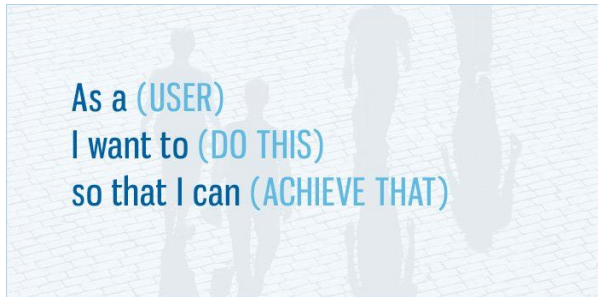


User Story ~= Feature

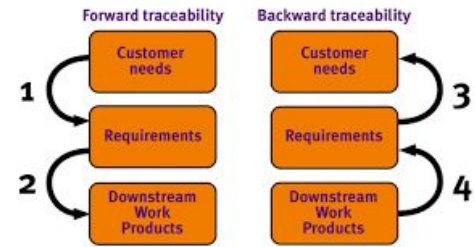
User stories define informal requirements in terms of what users will be able to do with the software that they couldn't do without it, or couldn't do as well without it, that provides them some *value*

A user story is a short simple description of ONE feature, ideally fitting on a 3x5 card, written in the user's language (domain terminology)

< label >: As a < type of user >, I want < some goal > so that < some reason >



User Story Elements



Why label? Useful for cross-referencing, e.g., [requirements tracing](#)

Why type of user? Software often has multiple *roles* that use the same program differently, such as performing different tasks (e.g., buyer vs. seller, doctor vs. patient, instructor vs. TA vs. student)

- ❖ In the case of client apps for a service, different roles do not imply different clients - the same client often has many different user roles. For example, Courseworks has 13 roles, but it's one client (with around 40k instances) not 13 different clients. However, Vergil is a different client that uses the same registrar service

Why goal? What the user wants, what the feature is supposed to do for the user

Why reason? Why the user wants it, how the feature will provide value to the user

Example User Stories



As a Backup Store user, I want to backup my entire hard drive so that my files are safe

As a Backup Store user, I want to specify which folders to backup to make sure my most important files are saved

As a Backup Store user, I want to specify subfolders and filetypes to skip within the backup folders so my backup store space isn't filled with things I don't need

(for a shared backup store) As a Backup Store administrator, I want to specify quotas and priorities for my users to manage the limited capacity of the backup store

Turning User Stories into Software

What other information might the developers need to start development?

Another way to think of it is what additional information will the backup program itself need?

- What permissions will the backup program need to access the files its supposed to backup
- Where to put the backup copies of the files
- What to do if the backup store is offline
- What to do if the backup store is full
- What to do with the files already in the backup store



SMART Conditions of Satisfaction

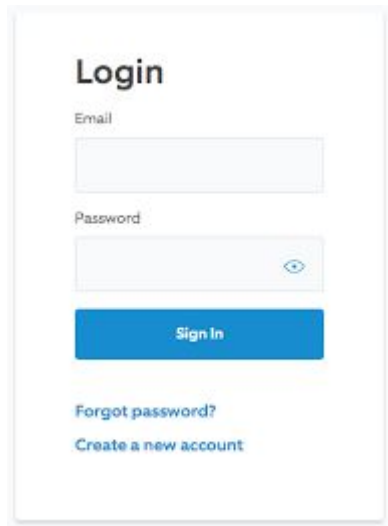


Conditions of satisfaction (acceptance criteria) ideally fit on the back of the 3x5 card, also written in user's language

< label >: As a < type of user >, I want < some goal > so that < some reason >

My conditions of satisfaction are < list of common cases and special cases that must work >

Do These Conditions of Satisfaction Say Enough to Start Development?



The image shows a login form with the following elements:

- Login** (Section Header)
- Email** (Label) with a text input field.
- Password** (Label) with a text input field and a toggle icon (eye) to the right.
- Sign In** (Button) in a blue box.
- [Forgot password?](#) (Link)
- [Create a new account](#) (Link)

As a user, I need to login before using the app

Conditions of Satisfaction:

- user is logged in only when proper credentials are entered
- user can request assistance
- user is locked out after three failed attempts

What other information might the developers need?

How would they know what their tests should check?

What About These?

As a vice president of marketing, I want to select a holiday season to be used when reviewing the performance of past advertising campaigns so I can identify the most profitable ones

Conditions of Satisfaction:

- Make sure it works with major retail holidays: President's Day, Memorial Day, Independence Day, Labor Day, etc.
- Support holidays that span two calendar years (Christmas through New Years).
- Support holiday seasons spanning one holiday to the next (Thanksgiving to Christmas).
- Holiday seasons can be set to be a number of days before/after the actual holiday.

What other information might the developers need?

How would they know what their tests should check?



Are There User Stories For Services?

There's no point in developing a service unless you have some ideas about what kinds of applications and other services (clients) might use it and what they would use it for - so there needs to be some way to describe the requirements for the API

Services are built for software engineers to call from their own software, not for end-users, so typically API requirements would be written in some API-oriented format using some API-oriented tool, like [OpenAPI](#) or [Swagger](#)

But we can still adopt user story format and domain terminology for informal requirements

And while user stories for applications should describe functionality rather than the data needed to implement that functionality, avoiding CRUD (create, read, update, delete), CRUD might be exactly what is needed from some service APIs



Possible Stories for a Shopping Service

As a visitor shopping app, I want an API that returns the list of nearby shops when I send a location, so that I can refer to these shops



Conditions of Satisfaction:

- The API should accept latitude, longitude and (optional) max-distance
- It should return a list of shops whose entrances are within max-distance (or default) of the requested lat-lon



As a rewards app, I want an API that returns the rewards programs affiliated with a specified shop, so that I can determine whether that shop is relevant to my users

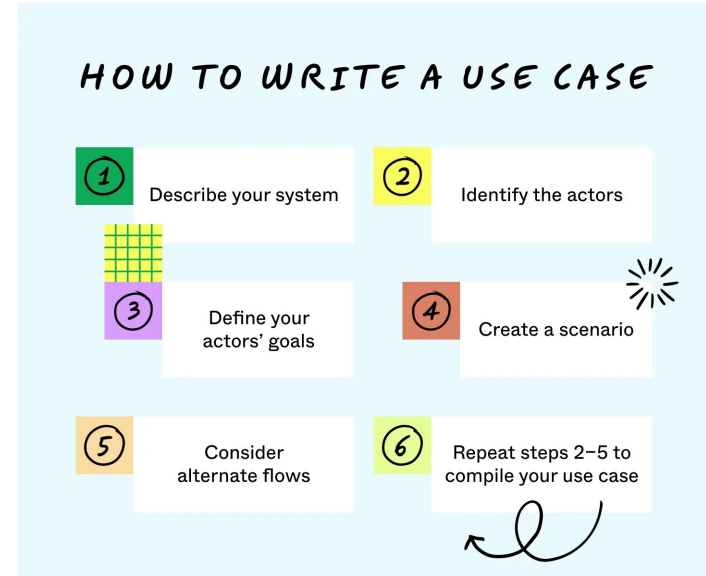
Conditions of Satisfaction:

- The API should accept the name and address of a specific shop
- It should return the list of rewards programs the shop participates in

➤ Notice the service does not itself support shopping, it supports shopping-related apps

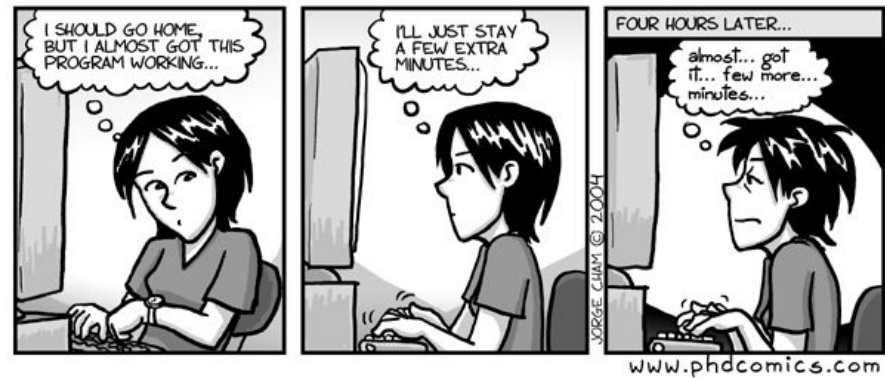
Agenda

1. An Ending and a Beginning
2. Upstream Software Engineering
3. User Stories
4. Use Cases
5. Reminders



Use Cases

It can be hard to extract enough information from user stories and their conditions of satisfaction to start development for a new project



Use cases expand user stories (or are written from scratch) to describe the step by step details of how a role interacts with the software to exercise the feature and achieve the goal, while trying to capture additional structure, contingencies and alternative flows, and (in the extreme) anything/everything that **CAN GO WRONG**

The term “*use case*” is often used by business to mean “a specific situation in which a product or service could potentially be used”, but here we mean more specifically “description of how an actor who actually uses that process or system will accomplish a goal”. The actor does not need to be human, could be client code

What is the Difference between User Story and Use Case?

User stories are about needs. When you write a user story, what you're describing is a “raw” user need. It's something that the user needs to do (or developer needs to use). If you never build any software for them, then that need will still exist! Written in domain language, so any member of the target user group should be able to understand what it means

Use cases are about the behavior you'll build into the software to meet those needs. A developer who needs to build working software should be able to read a use case and get a good sense of what the software needs to do. It typically has a lot of detail, and describes everything that the developer needs to build in order to meet the need. That's why it has a lot more detail, and needs to be clear and unambiguous



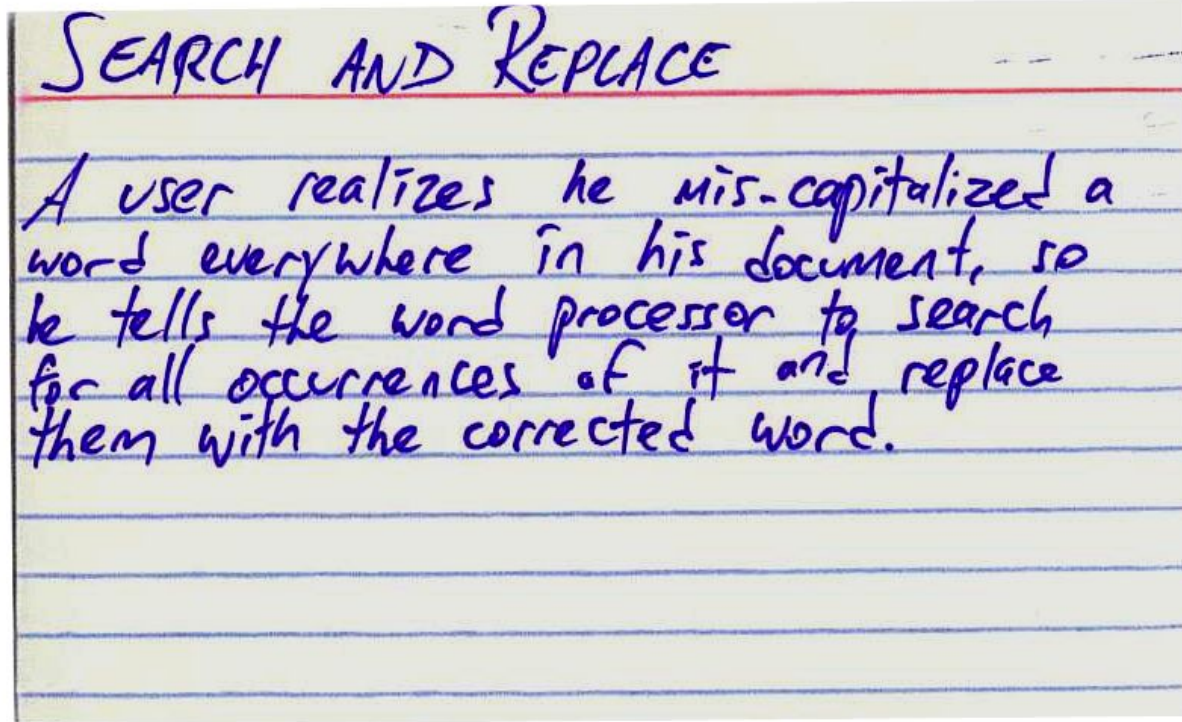
What Could Possibly Go Wrong?

Another difference between user stories and use cases is use cases might not use domain language, particularly for exception flows, since there are things that can go wrong that a user would not think about

- Invalid input data
- Invalid output data
- Some resource the feature relies on is unreachable, down, busy, slow, ...
- Request exceeds limits
- ...



Example User Story



Corresponding Use Case part 1

Name UC-8: Search and Replace

Summary All occurrences of a search term are replaced with replacement text.

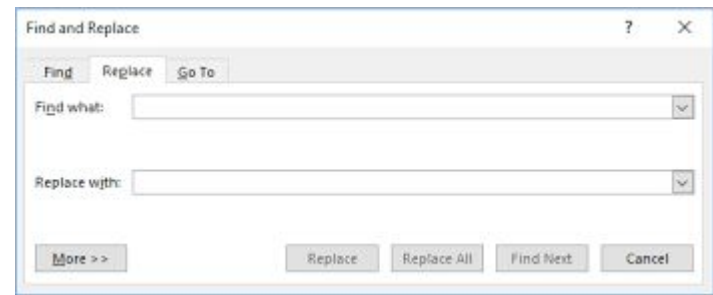
Rationale While editing a document, many users find that there is text somewhere in the file being edited that needs to be replaced, but searching for it manually by looking through the entire document is time-consuming and ineffective. The search-and-replace function allows the user to find it automatically and replace it with specified text. Sometimes this term is repeated in many places and needs to be replaced. At other times, only the first occurrence should be replaced. The user may also wish to simply find the location of that text without replacing it.

Actors All users

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.

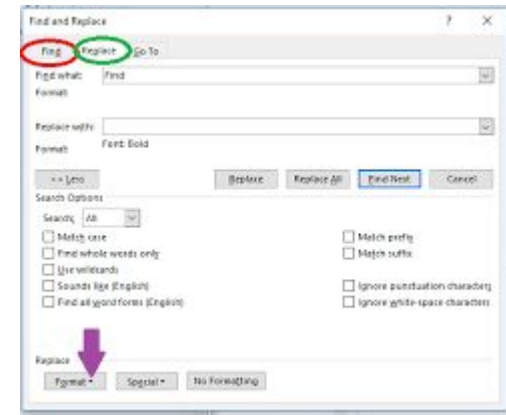


Corresponding Use Case part 2

...

Alternative Paths

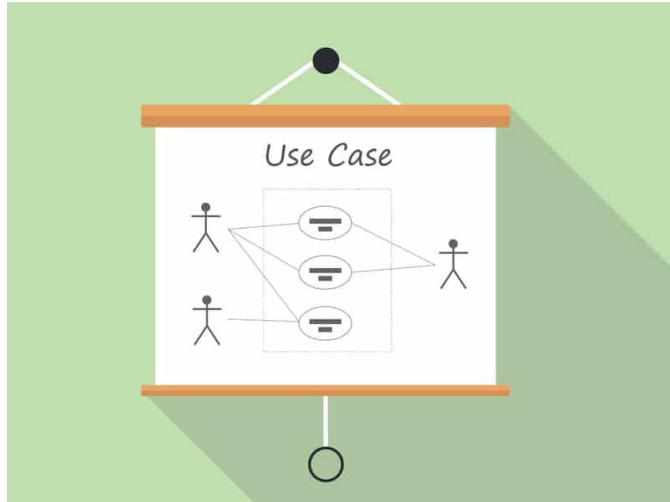
1. In Step 3, the user indicates that only the first occurrence is to be replaced. In this case, the software finds the first occurrence of the search term in the document being edited and replaces it with the replacement text. The postcondition state is identical, except only the first occurrence is replaced, and the replacement text is highlighted.
2. In Step 3, the user indicates that the software is only to search and not replace, and does not specify replacement text. In this case, the software highlights the first occurrence of the search term and the use case ends.
3. The user may decide to abort the search-and-replace operation at any time during Steps 1, 2, or 3. In this case, the software returns to the precondition state.



Postconditions All occurrences of the search term have been replaced with the replacement text.

Not UML Use Case Diagrams

If you google for “use case”, you may find something like this



We mean something like this

UC-0015	Register Book Loan	
Dependencies	• OBJ-0001 To manage book loans (objective) • OBJ-0005 To know library users' preferences (objective) • CRQ-0003 Maximum number of simultaneous loans (business rule) • CRQ-0014 Return date for a loan (business rule)	
Description	The system shall behave as described in the following use case when a library user requests a loan of one or more books.	
Precondition	The library user has been identified by means of his or her identity card, has picked up the books to loan from the shelves, has not reached the maximum number of simultaneous loans and has no penalty.	
Ordinary Sequence	Step	Action
	1	Actor librarian requests the system for starting the book loan registering process.
	2	The system requests for the identification of the library user requesting a loan.
	3	Actor librarian provides identification data of the library user to the system.
	4	The system requests for the identification of the books to be loaned.
	5	Actor librarian provides identification data of the books to be loan to the system.
	6	The system displays the return date for each of the books to be loan and requests loan confirmation for each of them.
	7	Actor library user confirms the librarian which books he or she wants to loan after knowing return dates.
	8	Actor librarian re-confirms the book loans confirmed by the library user to the system.
	9	The system informs that the book loans have been successfully registered.
Postcondition	The library user can take the loaned books away and the system has registered the book loans.	
Exceptions	Step	Action
	3	If the library user has already reached the maximum number of simultaneous loans or has a penalty, the system informs of the situation, then this use case is cancelled.
Comments	The maximum number of simultaneous book loans and the loan period depend on the library policy and can change in the future. See business rules CRQ-0003 y CRQ- 0014.	

Use Cases are More Structured than User Stories

Depending on how in-depth and complex you want or need to get, use cases describe a combination of the following elements:

- **Actor** – anyone or anything that performs a behavior (who is using the system)
- **Stakeholder** – someone or something with vested interests in the behavior of the system
- **Primary Actor** – stakeholder who initiates an interaction with the system to achieve a goal
- **Preconditions/Postconditions** – what must be true or happen before and after the use case runs
- **Triggers** – this is the event that causes the use case to be initiated
- **Main success scenario(s)** – aka basic flow or happy path
(not necessarily literally a single execution path)
- **Alternative flows** – special cases
- **Exception flows** – alternative flows for error cases

[Use case templates](#)



Example Use Case

1. On Wednesdays, the housekeeper reports to the laundry room.
2. The housekeeper sorts the laundry that is there.
3. Then they wash each load.
4. They dry each load.
5. A. They fold the items that need folding. $\Leftarrow$ basic flow
B. They iron and hang the items that are wrinkled. $\Leftarrow$ alternative flow
C. They throw away any laundry item that is irrevocably shrunken, soiled or scorched. $\Leftarrow$ alternative flow



Actors: Housekeeper, system that transports laundry to the laundry room.

Stakeholders: Owners/wearers of the laundry items.

Trigger: Dirty laundry has been transported to the laundry room.

Precondition: It is Wednesday.

Postcondition: Clean laundry is folded or hung. Damaged laundry is discarded.

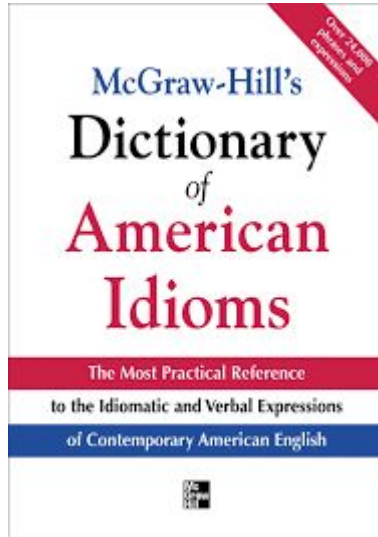
Possible Exception Flows



1. A. The housekeeper reports to the laundry room.
 B. The housekeeper does not show up. ⇐ **exception flow**
 C. The laundry room is flooded (or other reasons the housekeeper cannot get there). ⇐ **exception flow**
2. A. The housekeeper sorts the laundry that is there into hot, warm, cold wash (or other categories). ⇐ **more detail needed**
 B. The laundry is missing. ⇐ **exception flow**
3. A. Then they wash each load (in what order? what detergent, etc. is placed in washer?). ⇐ **more detail needed**
 B. The washing machine does not work. ⇐ **exception flow**
 C. The xxx load is too large for the washing machine and needs to be split (etc). ⇐ **more detail needed / exception**
 D. Washing machine takes too long to do all the loads before the housekeeper has to leave. ⇐ **exception flow**
4. And so on....

Aren't You Glad We Haven't Made You Write These?

Don't rejoice too soon, this material is fair game for the second assessment



Agenda

1. An Ending and a Beginning
2. Upstream Software Engineering
3. User Stories
4. Use Cases
5. Reminders



First Assessment This Weekend



Available this Friday October 25, 12:01AM (~= Thursday night)

Due Monday October 28, 11:59PM

See last Thursday's lecture (Oct 17) for details about the ungraded questions

Submission: You must submit a pdf file on courseworks. You can submit as many times as you like until the deadline. We strongly recommend submitting a draft early

Late Policy: Submissions posted within one day late (which includes everything courseworks marks as 1 minute late up to 23 hours 59 minutes late) will receive maximum 75% credit; within two days late maximum 50% credit; within three days late maximum 25% credit; more than three days late 0 credit

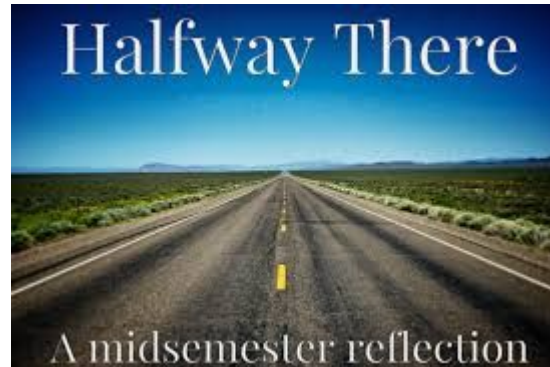
Rest of Semester Milestones

[Second Iteration](#) due November 25 (Monday before Thanksgiving)

[Second Iteration Demo](#) due December 6 (Friday a week after Thanksgiving)

Second Assessment Friday December 6 12:01AM to Monday December 9 11:59PM (last day of classes and the day before university "study days")

[Demo Day](#) Monday December 16 10AM to 4PM (Monday of finals week)



Thursday

Presentations/demos about including AI in software systems

[Including AI-based functionality in your second-iteration client and/or service is encouraged.

Using AI to write/test/document your code is not allowed.]



Ask Me Anything

